

Instalarea Java

⌘ Instalarea kit-ului JDK 1.8.x se realizeaza implicit in directorul:

⌘ **C:\Program Files\Java\jdk1.8.x**

care va contine subfolderele: **bin, demo, include, jre, lib** si **sample**.

⌘ In **bin** se gasesc principalele instrumente ale pachetului **JDK**:

- | | |
|---------------------------|--|
| ⊞ javac.exe | – compilator Java ; |
| ⊞ java.exe | – interpretor Java ; |
| ⊞ appletviewer.exe | – instrument de vizualizat applet -uri; |
| ⊞ javadoc.exe | – generator de documentatii; |
| ⊞ jar.exe | – instrument pentru arhive jar ; |
| ⊞ javap.exe | – dezasamblor de fisiere bytecode |

⌘ Crearea unei aplicatii simple

⌘ În principiu, există 3 tipuri de aplicații Java:

- ☒ a) aplicații de sine stătătoare (stand-alone);
- ☒ b) aplicații care se execută pe partea de client (applet-uri);
- ☒ c) aplicații care se execută pe partea de server (servlet-uri)

⌘ Realizarea unui program **JAVA** constă în următorii pași:

- ☒ Editarea programului într-un editor de texte;
- ☒ Salvarea programului sub numele **NumeClasa.java** unde **NumeClasa** este numele clasei care conține metoda **main()**. Într-un program **Java** trebuie să existe o singură clasă care să conțină o metodă **main()**. Cu alte cuvinte, numele clasei trebuie să coincidă cu numele fișierului. Extensia fișierului este **.java**
- ☒ Compilarea programului se face cu ajutorul comenzii
`javac NumeClasa.java`
- ☒ Executarea programului se face cu ajutorul comenzii
`java NumeClasa`

Crearea unei aplicatii simple

(continuare)

Scrierea codului sursă:

```
class Salut {  
    public static void main(String args[]) {  
        System.out.println("Salut !!!");  
    }  
}
```

- ⌘ Toate aplicațiile **Java** conțin o clasă principală în care trebuie să se găsească metoda **main()**. Clasele aplicației se pot găsi fie într-un singur fișier, fie în mai multe.

Crearea unei aplicatii simple

(continuare)

⌘ Salvarea fișierelor sursă

- ☒ Se va face în fișiere cu extensia **.java**. Fișierul care conține codul sursă al clasei principale trebuie să aibă același nume cu clasa principală a aplicației (clasa care conține metoda **main**). Prin urmare, fișierul nostru o să-l salvăm sub numele: **Salut.java**

⌘ Compilarea aplicației

- ☒ Se folosește compilatorul Java, **javac**. Apelul compilatorului se face pentru fișierul ce conține clasa principală a aplicației. Compilatorul creează câte un fișier separat pentru fiecare clasă a programului; acestea au extensia **.class** și sunt plasate în același director cu fișierele sursă. Rezultatul comenzii

javac Salut.java

este fișierul **Salut.class**

Crearea unei aplicatii simple

(continuare)

⌘ Rularea aplicației

- ☒ Se face cu interpretorul **Java**, apelat pentru unitatea de compilare corespunzătoare clasei principale, fiind însă omisă extensia **.class** asociată acesteia.

java Salut

- ☒ Rularea unei aplicații care nu folosește interfață grafică, se va face într-o fereastră sistem.

Structura lexicală a limbajului

Setul de caractere

- ⌘ Limbajului Java folosește setul de caractere **Unicode**. Este un standard internațional care înglobează setul de caractere **ASCII** (permite reprezentarea a 256 de caractere). Folosește pentru reprezentarea caracterelor 2 octeți, ceea ce înseamnă că se pot reprezenta 65536 de semne. Primele 256 caractere **Unicode** corespund celor din **ASCII**. Referirea la un caracter se face prin **\uxxxx**, unde **xxxx** reprezintă codul caracterului.

- ⌘ **Example:**

- ☒ \u0030 - \u0039 : cifre ISO-Latin 0 - 9
- ☒ \u0660 - \u0669 : cifre arabic-indic 0 - 9
- ☒ \u4e00 - \u9fff : litere din alfabetul Han (Chinez, Japonez, Coreean)

Constante

- ⌘ Constantele pot fi de următoarele tipuri

Constante întregi

- ⌘ Sunt acceptate 3 baze de numerație : baza 10, baza 16 (încep cu caracterele 0x) și baza 8 (încep cu cifra 0) și pot fi de două tipuri:
 - ⊗ normale, (se reprezintă pe 4 octeți - 32 biți)
 - ⊗ lungi (8 octeți - 64 biți): se termină cu caracterul L (sau l).

Constante reale

- ⌘ Pentru ca o constantă să fie considerată reală ea trebuie să aibă cel puțin o zecimală după virgulă, să fie în notație exponențială sau să aibă sufixul F sau f pentru valorile normale (reprezentate pe 32 biți), respectiv D sau d pentru valorile lungi (reprezentate pe 64 biți).

Constante logice

⌘ **true** : valoarea booleană de adevăr

⌘ **false** : valoarea booleană de fals

Observație: spre deosebire de **C++**, constantele întregi **1** și **0** nu mai au rolul de adevărat și fals.

Constante caracter

O constantă de tip caracter este utilizată pentru a exprima caracterele codului Unicode. Reprezentarea se face fie folosind o literă, fie o secvență **escape** scrisă între **apostrofuri**. Secvențele **escape** permit reprezentarea caracterelor care nu au reprezentare grafică și reprezentarea unor caractere speciale precum backslash, apostrof, etc.

Cod	Secvența Escape	Caracter
\u0008	'\b'	Backspace(BS)
\u0009	'\t'	Tab orizontal (HT)
\u000a	'\n'	'Linie nouă - linefeed (LF)
\u000c	'\f'	Pagină nouă - formfeed (FF)
\u000d	'\r'	Început de rând (CR)
\u0022	'\"'	Ghilimele
\u0027	'\''	Apostrof
\u005c	'\\'	Backslash

Constante șiruri de caractere

Un șir de caractere este format din zero sau mai multe caractere cuprinse între ghilimele. Caracterele care formează șirul de caractere pot fi caractere grafice sau secvențe **escape**. Dacă șirul este prea lung el poate fi scris ca o concatenare de subșiruri de dimensiune mai mică.

Concatenarea șirurilor se face cu operatorul + ("Ana " + "are " + "mere "). Șirul vid este "". După cum vom vedea, orice șir este de fapt, o instanță a clasei **String**, definită în pachetul java.lang.

Separatori

Un separator este un caracter care indică sfârșitul unei unități lexicale și începutul alteia. În Java separatorii sunt următorii: () { } [] ; , . Instrucțiunile unui program se separă cu ";".

Operatori

⌘ **operator de atribuire:** = (semnul egal)

Exemplu: a=9 (lui a i se atribuie valoarea 9)

Operatorii de atribuire pot fi înlanțuiți.

De exemplu: a=b=c=10

⌘ **operatori aritmetici binari:** +, -, *, /, %

Exemplu: s=a+b

În Java există forme prescurtate care cuprind operatorul de atribuire și un operator aritmetic binar. Operatorii prescurtați sunt: +=, -=, *=, /=, %=

Exemplu: n += 2 este echivalentă cu n=n+2

⌘ operatori aritmetici unari:

- ☐ +, -,
- ☐ ++ (operator de incrementare),
- ☐ -- (operator de decrementare).

Operatorii de incrementare și decrementare pot fi prefixați (++x sau --x) sau postfixați (x++ sau x--). Diferența dintre operatorii prefixați și cei postfixați este semnificativă doar atunci când expresia de incrementare / decrementare apare în cadrul unei expresii. Exemplele următoare vor evidenția aceste lucruri.

a) $-x$ reprezintă opusul lui x

b) `int x=5,y=7;`

`x++;` // x primește valoarea 6

`y - -;` // y primește valoarea 6

c) `int x=5,y=7;`

`++x;` // x primește valoarea 6

`- - y;` // y primește valoarea 6

d) `int x=5,y;`

`y=x++;` // y primește valoarea 5, x primește valoarea 6

e) `int x=5,y;`

`y=x - -;` // y primește valoarea 5, x primește valoarea 4

f) `int x=5,y;`

`y=++x;` // x primește valoarea 6, y primește valoarea 6

g) `int x=5,y;`

`y= - - x;` // x primește valoarea 4, y primește valoarea 4

⌘ **operatori logici:** &&(and), ||(or), !(not)

Observație: evaluarea expresiilor logice se face prin *scurtcircuitare* (evaluarea se oprește în momentul în care valoarea de adevăr a expresiei este sigur determinată)

⌘ **operatori relaționali:** <, <=, >, >=, ==, !=

⌘ **operatori pe biți:** & (and), |(or), ^(xor), ~(not)

⌘ **operatori de translație:** <<, >>, >>> (shift-are la dreapta fără semn)

⌘ **operatorul condițional:** "? : " . Are forma:

expresie_logica ? expresie1 : expresie2

Valoarea expresiei este dată de expresie1 dacă expresie_logică este true sau de expresie2 dacă expresie_logică este false.

Exemplu: Metoda de calculul minimului a două numere este:

```
public int min (int a, int b){  
    return ((a)<(b)) ? (a) : (b);  
} // la apel min(x+5,y-10)
```

⌘ **operatorul** , (virgula) este folosit pentru evaluarea secvențială a operațiilor

Exemplu: int x=0, y=1, z=2; (x=1,y=a=10,c=1)

⌘ **operatorul +** pentru concatenarea șirurilor:

```
String s="abcd";  
int    x=100;
```

```
System.out.println(s + " - " + x); //afiseaza abcd - 100
```

⌘ **operatorul de conversie de tip** (cast) este:
(tip_de_data)

```
int i    = 200;  
long l   = (long)i;      //conversie prin extensie  
long L2  = (long)200;  
int i2   = (int) L2;     //conversie prin contractie
```


⌘ *Exemplu:*

```
public class Operator{
    public static void main(String args[]){
        int a=2, b=3,c;
        c=a+b;      System.out.println(a+" "+b+" "+c);
        a++; --b;    System.out.println(a+" "+b+" "+c);
        c=++a+b++;  System.out.println(a+" "+b+" "+c);
        a/=2;b*=2;   System.out.println(a+" "+b+" "+c);
        c=a-- + --b;System.out.println(a+" "+b+" "+c);
        c=b=(a+=1); System.out.println(a+" "+b+" "+c);
    }
}
```

Comentarii



În Java există trei feluri de comentarii:

Comentarii pe o singură linie: încep cu `//`.

Comentarii pe mai multe linii, închise între `/*` și `*/`.

Comentarii pe mai multe linii care formează documentația, închise între `/**` și `**/`. Textul dintre cele două secvențe este automat mutat în documentația aplicației de către generatorul automat de documentație javadoc.



Observații:

nu pot fi scrise comentarii în interiorul altor comentarii.

nu pot fi introduse comentarii în interiorul constantelor caracter sau șir de caractere.

secvențele `/*` și `*/` pot să apară pe aceeași linie cu secvența `//` dar își pierd semnificația;

la fel se întâmplă cu secvența `//` în comentarii care încep cu `/*` sau `/**`.

Tipuri de date

- ⌘ În **Java** tipurile de date se împart în două categorii:
 - tipuri primitive de date**
 - tipuri referință.**
- ⌘ **Java** pornește de la premiza că "orice este un obiect". Prin urmare, tipurile de date ar trebui să fie de fapt definite de clase și toate variabilele ar trebui să memoreze de fapt instanțe (obiecte) ale acestor clase. În principiu acest lucru este adevărat, însă, pentru ușurința programării, mai există și așa numitele **tipuri primitive** de date, care sunt cele uzuale:

⌘ tipuri întregi

Tip de date	Dimensiune în octeți	Domeniu
byte	1	-128 .. 127
short	2	-32768 .. 32767
int	4	-2147483648 .. 2147483647
long	8	$-2^{63} .. 2^{63}-1$



⌘ tipuri reale

Tip de date	Dimensiune în octeți	Domeniu
float	4	$-10^{46} .. 10^{38}$
double	8	$-10^{324} .. 10^{308}$

- 
- ⌘ **tipul caracter**: char memorat pe 2 octeți
 - ⌘ **tipul boolean**: are două valori – **true** și **false**
 - ⌘ În alte limbaje formatul și dimensiunea tipurilor primitive de date folosite într-un program pot depinde de platforma pe care rulează programul. În Java acest lucru nu mai este valabil, Java fiind independent de platformă.
 - ⌘ Vectorii, clasele și interfețele sunt **tipuri referință**. Valoarea unei variabile de acest tip este, o referință (adresă de memorie) către valoarea sau mulțimea de valori reprezentată de variabila respectivă.
 - ⌘ Există trei tipuri de date C care nu sunt suportate de limbajul Java: pointer, struct și union. Pointerii au fost eliminați din cauza că erau o sursă constantă de erori, locul lor fiind luat de tipul referință, iar struct și union nu își mai au rostul atât timp cât tipurile compuse de date sunt formate în Java prin intermediul claselor.

Variabile

⌘ Variabilele pot avea ca tip fie un tip primitiv de dată, fie o referință la un obiect.

⌘ **Declararea variabilelor** se face prin:

```
tip_de_date  NumeVariabila
```

⌘ **Inițializarea variabilelor** se face prin:

```
NumeVariabila = valoare
```

⌘ Declararea și inițializarea variabilelor pot fi făcute în același moment:

```
tip_de_date  NumeVariabila = valoare
```

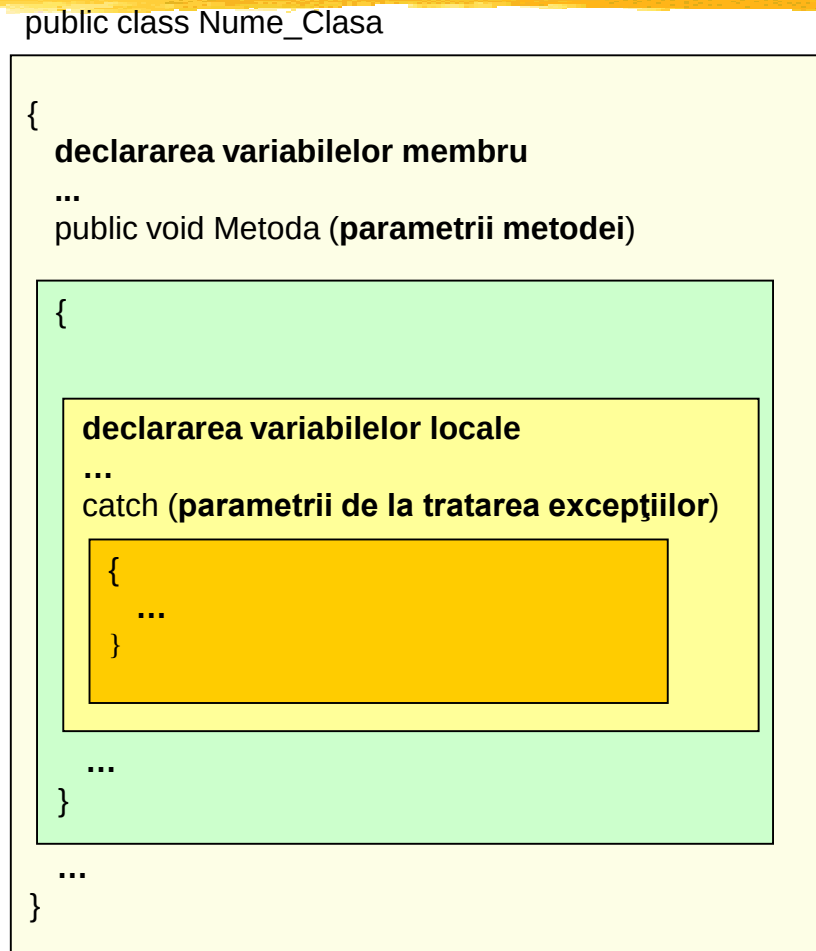
⌘ **Declararea constantelor** se face prin:

```
final tip_de_date  NumeVariabila
```

Exemplu: `final double PI = 3.14;`

- 
- ⌘ În funcție de locul în care sunt declarate, variabilele se împart în următoarele categorii:
 - ⌘ **Variabile membru**, declarate în interiorul unei clase, vizibile pentru toate metodele clasei respective și pentru alte clase în funcție de modifierul lor de acces
 - ⌘ **Variabile locale**, declarate într-o metodă sau într-un bloc de cod, vizibile doar în metoda / blocul respectiv
 - ⌘ **Parametrii metodelor**, vizibili doar în metoda respectivă
 - ⌘ **Parametrii de la tratarea excepțiilor**
- Imaginea următoare ilustrează tipurile de variabile și parametri împreună cu domeniul lor de vizibilitate.

⌘ Imaginea următoare ilustrează tipurile de variabile și parametri împreună cu domeniul lor de vizibilitate





Tablouri (vectori)

Tablouri unidimensionale

⌘ **Declararea** unui tablou se face prin

```
TipElement[] numeTablou; SAU
```

```
TipElement numeTablou[];
```

unde **TipElement** reprezintă tipul elementelor vectorului, iar parantezele [] așezate fie înaintea fie după numele vectorului arată că este vorba despre un vector.

⌘ *Exemple:*

```
int[] v;
```

```
String adrese[];
```

- 
- ⌘ **Instanțierea** unui vector se realizează cu operatorul **new** și are ca efect alocarea memoriei necesare pentru memorarea elementelor vectorului, mai precis specificarea numărului maxim de elemente pe care îl va avea vectorul. Instanțierea unui vector se face astfel:

```
numeVector = new TipElement[dimensiune];
```

- ⌘ *Exemple:*

```
v = new int[10];           //se alocă spațiu pentru 10 întregi  
adrese = new String[100]; //se alocă spațiu pentru 100 de String-uri
```

- ⌘ Declararea și instanțierea unui vector pot fi făcute simultan astfel:

```
TipElement[] numeVector = new TipElement[dimensiune];
```

- ⌘ *Exemple:*

```
int [] v = new int[10];    // asta va recomand
```

- 
- ⌘ După **declararea** unui vector, acesta poate fi **inițializat**, adică elementele sale pot primi valori. În acest caz **instanțierea lipsește**, alocarea memoriei făcându-se automat în funcție de numărul de elemente cu care se inițializează vectorul.

⌘ *Exemple:*

```
String culori[] = {"Rosu", "Galben", "Verde"};  
int []v = {2, 4, 6, 8, 10, 12};
```

⌘ *Observații:*

- Primul indice al unui vector este 0, deci pozițiile unui vector cu n elemente vor fi cuprinse între 0 și n-1.
- Nu sunt permise construcții de genul: **TipElement numeVector[dimensiune];** alocarea memoriei făcându-se doar prin intermediul operatorului **new** sau prin **inițializare**.

⌘ *Exemple:*

```
int v[10]; //incorect  
int v[] = new int[10]; //corect
```

⌘ **Accesul** la elementul unui vector se face prin:

```
numeVector[indice]
```

⌘ *Exemplu 1:*

```
int v[]=new int[10];  
for(i=0; i<10; i++)  
    v[i]=i;
```

⌘ *Exemplu 2:*

```
int v[]={1,2,3,4,5};    //corect!! este initializare  
for(i=0; i<5; i++)  
    System.out.println(v[i]+" *");
```

Tablouri cu mai multe dimensiuni

- ⌘ În Java tablourile cu mai multe dimensiuni sunt de fapt tablouri de tablouri. Prin urmare, declararea, instanțierea și inițializarea se fac la fel ca în cazul tablourilor unidimensionale.

```
TipElement numeTablou[][]... = new TipElement[dim1][dim2]...
```

sau

```
TipElement[][]... numeTablou = new[dim1][dim2]... TipElement  
(parantezele pot fi de o parte și de alta a lui numeTablou)
```

- ⌘ *Exemplu:*

```
int m[][];           //declararea unei matrice  
m = new int[5][10]; //cu 5 linii, 10 coloane
```

Observație:

`m[0]`, `m[1]`, ..., `m[4]` sunt tablouri de întregi cu 10 elemente

Dimensiunea unui tablou

⌘ Cu ajutorul cuvântului cheie **length** se poate afla dimensiunea unui tablou.

⌘ *Exemple:*

Fie vectorul

```
int []a = new int[5];
```

atunci **a.length** are valoarea 5.

⌘ Fie matricea

```
int m[][] = new int[5][10];
```

atunci:

m.length are valoarea 5 și reprezintă numărul de linii al matricei

m[0].length are valoarea 10 și reprezintă numărul de elemente al primei linii a matricei,

m[1].length are valoarea 10 și reprezintă numărul de elemente al celei de-a doua linii a matricei, etc.

⌘ *Exemplu:* Să se calculeze minimul elementelor unui tablou.

```
public class MinVect{  
    public static void main(String args[]){  
        int t[]={2,1,4,7,3};  
        int min=t[0];  
        for(int i=1; i<t.length; i++)  
            if (t[i]<min) min=t[i];  
        System.out.println("Minimul este "+min);  
    }  
}
```


⌘ Java permite folosirea tablourilor cu dimensiuni variabile adică, a tablourilor de tablouri cu dimensiuni variabile.

⌘ *Exemplu*: Să se genereze și să se afișeze triunghiul lui Pascal.

```
public class TrPascal{
    public static void main(String args[]){
        int [][]t[] =new int[10][];           //declarare matrice variabila
        t[0]          =new int[1];           //initializare primele 2 linii
        t[1]          =new int[2];
        t[0][0]       =t[1][0]=t[1][1]=1;

        for(int i=2; i<t.length; i++){        //generare triunghi
            t[i]=new int[i+1];
            t[i][0]=t[i][i]=1;
            for(int j=1; j<t[i].length-1; j++)
                t[i][j]=t[i-1][j-1]+t[i-1][j];
        }

        for(int i=0; i<t.length; i++){        //afisare triunghi
            for(int j=0; j<t[i].length; j++)
                System.out.print(t[i][j]+" ");
            System.out.println("");
        }
    }
}
```

Șiruri de caractere

⌘ În Java, un șir de caractere poate fi reprezentat printr-un tablou format din elemente de tip `char`, un obiect de tip `String` sau un obiect de tip `StringBuffer`.

⌘ *Exemple* echivalente de declarare a unui șir:

```
String str = "abc";  
char data[] = {'a', 'b', 'c'};  
String str = new String(data);  
String str = new String("abc");
```

⌘ **Concatenarea șirurilor** de caractere se face prin intermediul operatorului `+`.

```
String str1 = "abc" + "xyz";  
String str2 = "123";  
String str3 = str1 + str2;
```

⌘ În Java, operatorul de concatenare `+` este extrem de flexibil în sensul că permite concatenarea șirurilor cu obiecte de orice tip care au o reprezentare de tip șir de caractere.

⌘ *Exemplu:*

```
System.out.print("Tabloul t are" + t.length + " elemente");
```

Clase și obiecte în Java

Referințe

- ⌘ După cum am arătat, **o clasă** este un șablon pentru mai multe obiecte cu caracteristici asemănătoare. Un obiect este o colecție de variabile (atribute) și metode asociate descrise în clasă.
- ⌘ Clasa poate fi asemănată cu un tip de date iar obiectul cu o variabilă. Dacă se declară o variabilă folosind numele unei clase ca și tip, această variabilă conține o **referință** către un obiect al clasei respective. Cu alte cuvinte, variabila nu va conține obiectul actual ci o referință către un obiect / o instanță a clasei. Deoarece folosind numele unei clase ca și tip se declară o referință către un obiect, aceste tipuri poartă numele de **tipuri referință**.

- ⌘ Se disting două caracteristici principale ale obiectelor în Java:
- ⌘ obiectele sunt întotdeauna alocate dinamic. Durata de viață a unui obiect este determinată de logica programului. Ea începe atunci când obiectul este creat și se termină în momentul în care obiectul nu mai este folosit, el fiind distrus de un mecanism de curățenie oferit de limbajul Java – **garbage collector** (colectorul de gunoaie).
- ⌘ obiectele nu sunt conținute de către variabile. O variabilă păstrează o referință către un obiect. O referință este similară cu ceea ce se numește **pointer** în alte limbaje de programare cum ar fi C++. Dacă există două variabile de același tip referință și o variabilă este atribuită celeilalte, ambele vor referi același obiect. Dacă informația din obiect se modifică, schimbarea este vizibilă în ambele variabile.
- ⌘ O variabilă referință poate conține și o referință către nimic. Valoarea unei asemenea variabile referință este **null**.
- ⌘ În Java **nu se permite** tipurilor referință să fie convertite către tipuri primitive sau invers.

Obiecte. Noțiuni generale

⌘ În Java obiectele sunt create prin instanțierea unei clase, cu alte cuvinte prin crearea unei instanțe a unei clase.

⌘ **Declararea obiectului** se face prin:

```
NumeClasa numeObiect;
```

⌘ *Exemplu:*

```
String s;  
Complex c;
```

În urma declarării, variabila este inițializată cu null.

⌘ **Crearea obiectului** echivalentă cu instanțierea clasei se realizează prin intermediul operatorului **new** și presupune alocarea spațiului de memorie necesar păstrării obiectului. Adresa (referința) către obiectul respectiv o să fie memorată în variabila numeObiect.

```
numeObiect = new NumeClasa();
```

⌘ *Exemplu:*

```
s = new String() ;  
Complex c = new Complex(2,3);
```

- ⌘ Declararea și crearea obiectului pot fi făcute pe aceeași linie.

Exemplu:

```
String s=new String() ;  
Complex c=new Complex(2,3);
```

- ⌘ În momentul în care se realizează crearea obiectului are loc și inițializarea lui.

Atenție

Spațiul de memorie nu este alocat în momentul declarării obiectului.

- ⌘ *Exemplu:*

```
Complex c;c.re = 10;           //EROARE! Alocarea memoriei se face doar  
                               //la apelul instrucțiunii new !
```

- ⌘ **Referirea valorii unei variabile** se face prin
 obiect.variabila

⌘ *Exemplu:*

```
Complex c = new Complex();  
c.re      = 2;  
c.im      = 3;  
System.out.println("Re=" +c.re+ "   Im="+c.im);
```

⌘ *Observație:* Accesul la variabilele unui obiect se face în conformitate cu drepturile de acces pe care le oferă variabilele respective celorlalte clase.

⌘ **Apelul unei metode** se face prin
obiect.metoda([parametri])

⌘ *Exemplu:*

```
Complex c = new Complex();  
c.setRe(2);  
c.setIm(3);
```

Operatorul de atribuire =

- ⌘ În cazul în care operatorul = este folosit împreună cu date de tip primitiv, are loc o atribuire de valori.

Exemplu:

```
int x = 12, y = 10;  
x = y;           //Valoarea lui x se schimbă, primind valoarea lui y.
```

- ⌘ În cazul în care operatorul = este folosit împreună cu date de tip referință, are loc tot o atribuire de valori. Atâta doar că valorile sunt în acest caz adrese.

⌘ *Exemplu:*

```
class Nimic{  
    int x;  
}  
public class Test{  
    public static void main(String args[]){  
        Nimic a1=new Nimic(),a2;  
        a1.x=10;  
        a2=a1;  
        System.out.println(a1.x+" "+a2.x);  
        a1.x=5;  
        System.out.println(a1.x+" "+a2.x);  
    }  
}
```


Clase

Definirea claselor

```
⌘ [public][abstract][final] class NumeClasa  
    [extends NumeSuperclasa]  
    [implements Interfata1 [, Interfata2 ... ]]  
    {  
        //corpul clasei  
    }
```

⌘ Antetul clasei:

```
[public][abstract][final] class NumeClasa
```

⌘ este format din **modificatorii clasei**, cuvântul rezervat **class** și numele clasei **NumeClasa**.

⌘ Prezența parantezelor **[]** indică faptul că ceea ce este cuprins între ele este opțional.

⌘ **Modificatorii clasei** sunt:

- ⌘ **public**: dacă o clasă este declarată public, ea este accesibilă oricărei alte clase. Dacă modificatorul public nu apare, clasa poate fi accesată doar de către clasele din pachetul căruia aparține clasa (dacă nu se specifică un anumit pachet, toate clasele din directorul curent sunt considerate a fi în același pachet). Spațiul de căutare este definit de variabila sistem **CLASSPATH**.
- ⌘ **abstract**: o clasă declarată abstractă este o clasă șablon adică ea este folosită doar pentru a crea un model comun pentru o serie de subclase. O clasă trebuie declarată abstractă dacă ea este incomplet implementată adică nu toate metodele ei sunt definite. O astfel de clasă **nu poate fi instanțiată**, dar poate fi extinsă de alte clase care să implementeze metodele nedefinite. Doar clasele abstracte pot să conțină metode abstracte (metode declarate dar nu implementate).
- ⌘ **final** o clasă poate fi declarată finală dacă a fost complet definită și nu se dorește să fie extinsă (să aibă subclase); cu alte cuvinte ea nu poate apare în clauza **extends** a altei clase.

Variabile membru

- ⌘ Variabilele se declară de obicei înaintea metodelor, deși acest lucru nu este impus de compilator.

```
class NumeClasa {  
    //declararea variabilelor  
    //declararea metodelor  
}
```

- ⌘ Variabilele unei clase sunt doar cele care sunt declarate în corpul clasei și nu în corpul unei metode. Variabilele declarate în cadrul unei metode sunt locale metodei respective.
- ⌘ Declararea unei variabile presupune specificarea următoarelor lucruri:
 - numele variabilei;
 - tipul de date;
 - nivelul de acces la acea variabilă de către alte clase;
 - dacă este constantă sau nu;
 - tipul variabilei: instanță sau clasă

⌘ Tiparul declarării unei variabile este:

`[modificatori] TipDeDate numeVariabila [= valoareInitiala];`

⌘ unde un modificador poate fi :

☑ un specificator de acces: **public, protected, private;**

☑ unul din cuvintele rezervate: **static, final, transient, volatile.**

⌘ *Exemple:*

☑ `double x;`

☑ `protected static int n;`

☑ `public String s = "abcd";`

☑ `private Point p = new Point(10, 10);`

☑ `final long MAX = 1000000L;`

⌘ Cuvintele rezervate: **static**, **final**, **transient**, **volatile** au următoarele roluri:

⌘ **static** este folosit pentru declararea variabilelor clasă.

Exemplu:

⌘ `int x ; //variabilă instanță`

⌘ `static int nrDeObiecteCreate; //variabilă clasă`

⌘ **final** este folosit pentru declararea constantelor. O constantă este o variabilă a cărei valoare nu mai poate fi schimbată. Valoarea unei variabile finale nu trebuie specificată neapărat la declararea ei, ci poate fi specificată și ulterior, după care ea nu va mai putea fi modificată.

⌘ *Exemplu:* se declară și se inițializează o variabilă finală (linia 1). Valoarea ei nu mai poate fi modificată (linia 3).

⌘ `final double PI = 3.14 ;`

⌘ `...`

⌘ `PI = 3.141 // eroare la compilare`

- ⌘ *Exemplu:* se declară o variabilă finală (linia 2). În linia 4 se inițializează iar în linia 5 se dorește modificarea valorii ei.

```
class Test {  
    final int MAX;  
    Test() {  
        MAX = 100;    // legal  
        MAX = 200;    // ilegal -> eroare la compilare  
    }  
}
```

- ⌘ **transient** este folosit la serializarea obiectelor, pentru a specifica ce variabile membru ale unui obiect nu participa la serializare.
- ⌘ **volatile** este folosit pentru a semnala compilatorului să nu execute anumite optimizări asupra membrilor unei clase. Este o facilitate avansată a limbajului Java.

Metode

Definirea metodelor

- ⌘ Metodele sunt folosite pentru descrierea comportamentului unui obiect. O metodă se declară astfel:

```
[modifieri] TipReturnat numeMetoda ([argumente])  
[throws TipExceptie]  
{  
    //corpul metodei  
}
```

Modificatorii metodelor

- ⌘ Un modifier poate fi :
- un specificator de acces: **public, protected, private;**
 - unul din cuvintele rezervate: **static, abstract, final, native, synchronized.**

⌘ Cuvintele rezervate: **static**, **abstract**, **final**, **native**, **synchronized** au următoarele roluri:

⌘ **static** este folosit pentru declararea metodelor clasă.

Exemplu:

```
void metoda1() ;           //metoda de instanta  
static void metoda2();    //metoda de clasa
```

⌘ **abstract** este folosit pentru declararea metodelor abstracte. O metodă abstractă este o metodă care nu are implementare și trebuie să aparțină unei clase abstracte.

- 
- ⌘ **final** este folosit pentru a specifica faptul că acea metodă nu mai poate fi supradefinită în subclasele clasei în care ea este definită ca fiind finală. Acest lucru este util dacă respectiva metodă are o implementare care nu trebuie schimbată în subclasele ei.
 - ⌘ **native** este folosit pentru refolosirea unor funcții scrise în alt limbaj de programare decât Java (C de exemplu).
 - ⌘ **synchronized** este folosit în cazul în care se lucrează cu mai multe fire de execuție iar metoda respectivă se ocupă cu partajarea unei resurse comune. Are ca efect construirea unui semafor care nu permite executarea metodei la un moment dat decât unui singur fir de execuție.

Exemple Java

⌘ *Exemplul 1:*

```
public class Maxim {           //determina maximul dintre 2 intregi
    public static void main(String args[]){
        int a=12, b=5,m;
        if (a>b)  m=a;
        else     m=b;
        System.out.println("Maximul dintre "+a+" si "+b+" este: "+m);
    }
}
```

Exemple Java

⌘ *Exemplul 2:*

```
public class Minim {    //determina minimul dintre 2 intregi
    public static void main(String args[])    {
        int a=-5, b=8;
        System.out.println("a="+a+" b="+b);
        System.out.println("minimul este:"+min(a,b));
        System.out.println("minimul este:"+ (a<b?a:b));
    }

    public static int min (int a, int b) {
        return a<b?a:b;
    }
}
```

Exemple Java

⌘ Exemplul 3:

```
import java.io.*;
import java.lang.Math.*;
import java.util.Scanner;
public class cmmdcDif{
    public static int CitLong (String sir){
        Scanner scn = new Scanner(System.in);
        System.out.print(sir);
        return scn.nextInt();
    }
    public static long cmmdcD(long a, long b){
        while (a!=b)
            if (a>b) a=a-b;
            else    b=b-a;
        return a;
    }
    public static void main(String args[]){
        long a,b;
        a=CitLong("da primul numar:");
        b=CitLong("da inca un numar:");
        System.out.print("Cmmdc-ul numerelor "+a+" si "+b+" este: ");
        System.out.println(cmmdcD(Math.abs(a),Math.abs(b)));
    }
}
```

Example Java

⌘ Exemplul 4:

```
public class EcGrad1{
    private double a;           // coeficientii ec. aX+b=0;
    private double b;
    public EcGrad1 (double a, double b){ // constructor
        this.a=a;
        this.b=b;
    }
    public void setA(double a){    // setari a si b
        this.a=a;
    }
    public void setB(double b){
        this.b=b;
    }
    public double  getA(){         // returnari a si b
        return a ;
    }
    public double  getB(){
        return b ;
    }
    public void afisare1(){        // metoda de afisare ar trebui scrisa in programul de aplicatie
        if (a!=0){                // si nu in clasa
            if (a!=1.0) System.out.print(getA()+"*X");
            else         System.out.print("X");
        }
        if(getB()<0)  System.out.print(getB()+"=0");
        else         System.out.println(" "+getB()+"=0");
    }
    public void rezolva1(){        // rezolvare
        if (a!=0)          System.out.println("ec. are sol. unica:"+ -b/a);
        else if (b!=0)     System.out.println("ec. nu are solutii");
        else               System.out.println("ec. are o inf. de solutii reale");
    }
}
```